NeonCore 5G AI Roadmap

Build plan for extending the NeonCore 5G testbed with AI. Week numbers are relative to the project start.

Where the project stands today

The 5G lab is done; what is missing is AI that has been built and measured. NeonCore 5G already produces the two things AI needs, labeled signaling traffic and live KPIs, so every project in this plan extends it instead of starting over.

Asset	Evidence	Why it matters for AI
5G SA core lab	Open5GS + UERANSIM on K3s, 13 pods, Multus, headless SCTP services	A safe environment to generate data and run agents against
Packet tracing	Tracer pod capturing N2, N4, N3 and SBI, rotated pcaps on a PVC	Raw material for a call-flow troubleshooting dataset
Observability	Prometheus + Grafana, AMF/SMF/UPF/PCF metrics, kube-state-metrics	Time series for KPI anomaly detection
Scenario automation	phase6: registration, registration reject, deregistration, one pcap per run	Already a labeled-data generator; it only needs more fault types
Engineering log	DEVLOG with root-caused bugs (WSL2 clock drift, TUN permissions)	A record of real failures that informs fault design

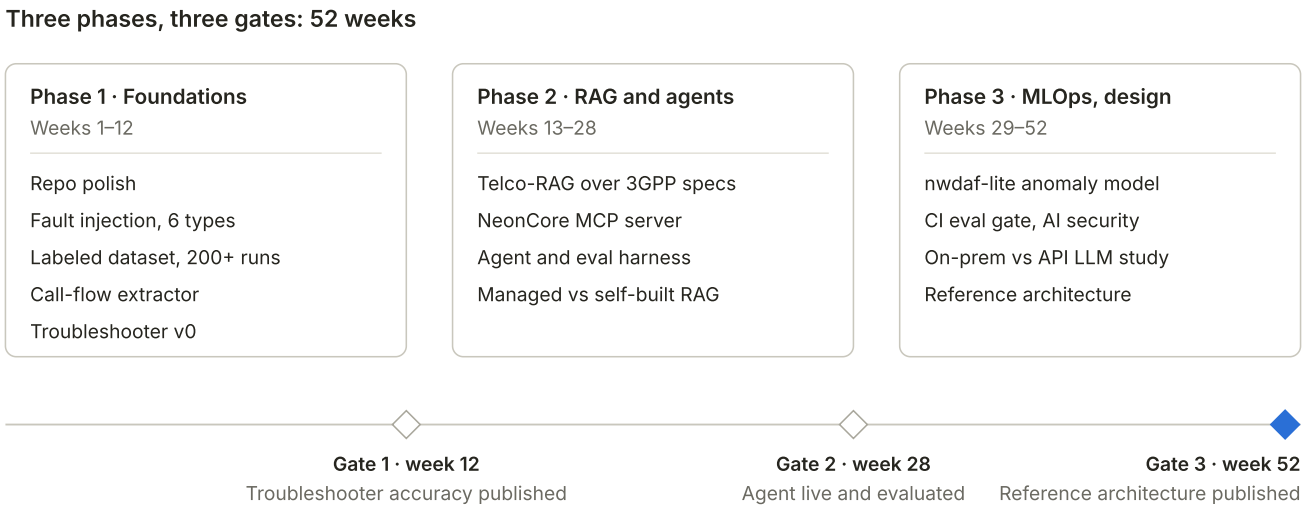
Gaps to close

- No AI component yet: no model, no RAG, no agent, no evaluation numbers.
- No training, metrics or evaluation design in the project.
- No MLOps or LLMOps: experiment tracking, model serving, drift, CI gates.
- Little project documentation: the repo has no description or topics, 3 commits and no write-ups.
- No AI security or governance work.

Project goal: build a 5G core, generate labeled faults, and build AI that finds them, with numbers to prove it.

Plan at a glance

Three phases of about 3, 4 and 6 months, each ending in a gate.



roadmap · 3 phases, 3 gates, 52 weeks

Don't start a phase until the previous gate is met; if a gate slips, move the schedule, not the order.

Phase 1: Foundations and labeled faults (weeks 1–12)

By week 12 NeonCore produces labeled faults and a first AI troubleshooter with a measured accuracy.

Weeks	Build in NeonCore	Done when
1–2	Repo polish: description, topics, demo GIF of the TUI, a "Roadmap" section in the README	Repo explains itself in 30 seconds
3–4	Phase 7 fault injection: wrong K/OPc (auth failure), unknown DNN, wrong S-NSSAI	3 faults reproducible from the CLI `s` menu
5–6	More faults: PCF down, SMF scaled to 0, UPF down; label each run	6 fault types, each with its own pcap
7–8	Dataset generator: scenario × fault × MSISDN loop writing pcaps, labels and a Prometheus snapshot to `datasets/` with a manifest CSV	At least 200 labeled runs
9–10	tshark extractor: pcap to JSON call flow (NGAP and NAS messages, 5GMM/5GSM cause codes, timings)	JSON matches Wireshark for every fault type
11–12	Phase 8 v0: LLM reads the JSON call flow and returns failing step, cause code and root cause; scored against the labels	Accuracy per fault type in a results table

Phase 1 checkpoints

- [ ] Repo polished
- [ ] 6 fault types reproducible on demand
- [ ] Labeled dataset of 200+ runs
- [ ] Troubleshooter v0 with an accuracy table in the README

Phase 2: RAG and agents (weeks 13–28)

By week 28 NeonCore has a spec-aware agent that diagnoses injected faults end to end. LLMs run through an API (Claude API or Amazon Bedrock), not locally.

Weeks	Build in NeonCore	Done when
13–14	Phase 9 Telco-RAG: ingest TS 23.501, 23.502, 24.501 (NAS) and 38.413 (NGAP), chunked by clause number	Questions return the right clause
15–16	Hybrid search and reranking; answers cite clause numbers; eval on a GSMA Open-Telco benchmark subset plus 50 project-specific questions	Scores recorded in the README
17–18	Phase 10 NeonCore MCP server: read-only tools for pods, logs, PromQL, pcap summary, run scenario	Claude Desktop or Claude Code can query the live core
19–20	Agent (LangGraph or similar, with guardrails and human-in-the-loop) that combines the troubleshooter, Telco-RAG and MCP tools; any write (restart, scale) needs approval; new TUI key `a`	Agent diagnoses a fault injected live
21–22	Agent eval harness: task success, correct tool calls, cost and latency per query	One-command eval run with a summary table
23–24	Port Telco-RAG to Bedrock Knowledge Bases and compare with the self-built version	Short decision record: managed vs self-built RAG
25–26	Demo video (5 minutes) of the agent finding a fault	Video linked from the README
27–28	Buffer for anything that slipped	Phase 2 checkpoints met

Phase 2 checkpoints

- [ ] Telco-RAG with published eval scores
- [ ] NeonCore MCP server in the repo
- [ ] Agent eval harness with a results table
- [ ] Managed-vs-self-built RAG decision record

Phase 3: MLOps, standards and architecture (weeks 29–52)

By week 52 the project has an NWDAF-style analytics function in production shape and a reference architecture. Open5GS has no NWDAF, so building one is a natural extension of NeonCore.

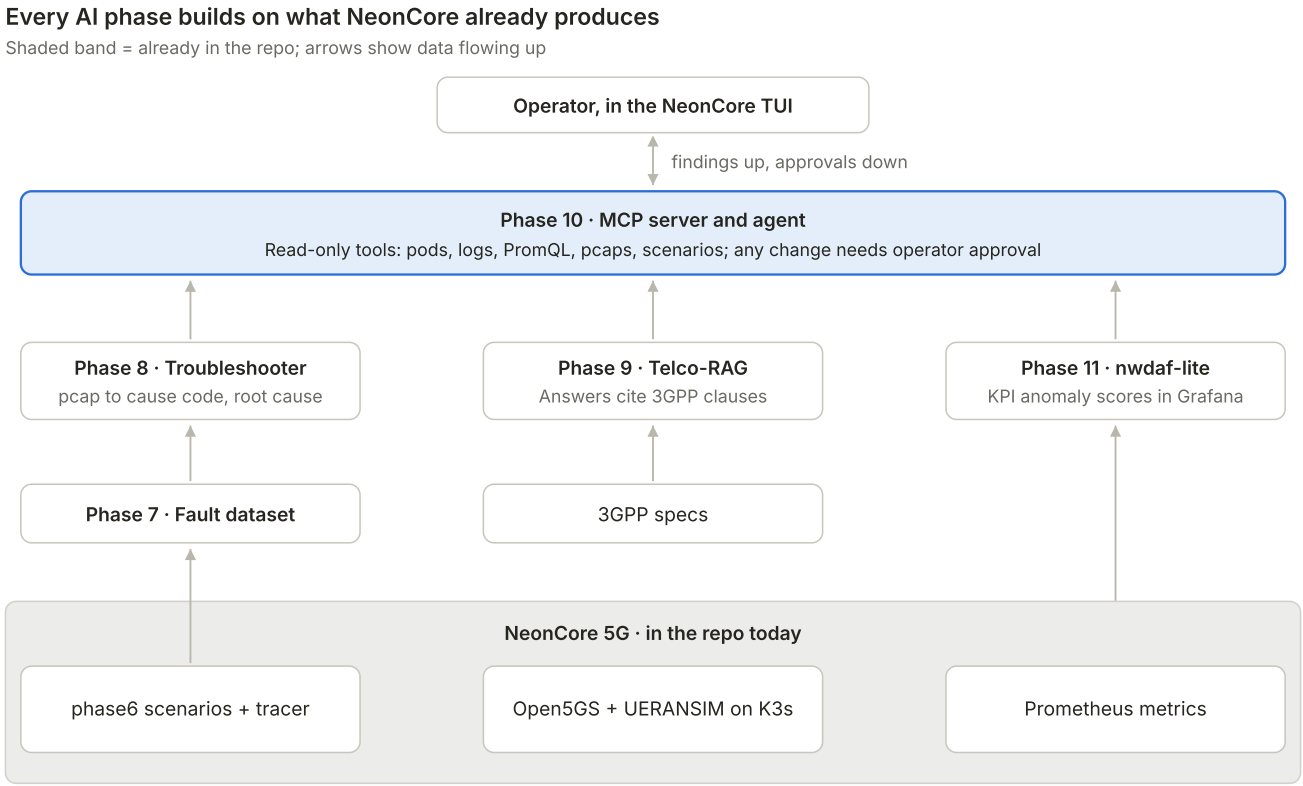
Weeks	Build in NeonCore	Done when
29–31	Phase 11 KPI dataset with MLflow tracking: AMF registrations, SMF PDU sessions, UPF GTP-U packet rate under normal load and each fault	Time-series dataset with fault windows labeled
32–34	`nwdaf-lite`: anomaly model (for example isolation forest or simple forecasters) served as a K8s Deployment with an analytics API; anomaly score panel in Grafana	Grafana flags the fault that was injected
35–36	GitHub Actions: lint, unit tests, eval gate that blocks a worse model	A failing eval blocks a merge
37–38	Map `nwdaf-lite` outputs to NWDAF analytics types (TS 23.288) in the README	A spec-to-code mapping table
39–40	Benchmark an open model (vLLM on a rented cloud GPU, a few hours only) vs the API on the eval set: quality, latency, cost per 1,000 queries	Decision record: on-prem vs API LLM for an operator
41–43	Red-team the agent against the OWASP Top 10 for LLM apps: a malicious string in a log or SBI field must not trigger a tool call	Injection tests in CI
44–47	Reference architecture: GenAI-assisted NOC for a Tier-1 operator, built from the lab results	Document published and linked from the README
48–52	Project landing README and 10-minute demo	Landing README and demo published

Phase 3 checkpoints

- [ ] `nwdaf-lite` detecting injected faults in Grafana
- [ ] CI with an eval gate and injection tests
- [ ] On-prem vs API LLM decision record
- [ ] Reference architecture published

Project specs built on NeonCore

Each project is a new phase folder in the same repo, so the work reads as one system that grows, not five demos. Every project ships with an evaluation number.



NeonCore AI architecture · phases 7 to 11

The agent sits on top and calls the three AI components as tools; it also reaches the live core through the MCP server, but never changes it without operator approval.

Design rules for all projects

- The agent reads freely but asks before any change, the same way the CLI already confirms deploy and teardown.
- Treat logs, pcaps and SBI payloads as untrusted input to the LLM.
- Record cost and latency next to accuracy; operators decide on all three.

Project ground rules

- One write-up per finished phase: the problem, a screenshot, the number, what broke.
- Commit in small steps with clear messages.
- Never use real operator pcaps, configs or IMSIs, or proprietary documents. NeonCore and synthetic data only.
- Set a monthly spending cap on API and cloud GPU accounts.

Projects, stack and proof

Phase	Project	Builds on	Stack	Proof it works
7	Fault injection and dataset generator	phase6 scenarios, tracer pod	Python, kubectl, Open5GS subscriber tools	6 fault types, 200+ labeled runs
8	AI call-flow troubleshooter	Phase 7 pcaps	tshark, Python, Claude API, structured output	Accuracy per fault type vs labels
9	Telco-RAG spec assistant	3GPP specs	Embeddings, vector store, hybrid search, reranker	Benchmark and project-question scores, clause citations
10	NeonCore MCP server and agent	Phases 8 and 9, Prometheus, CLI	MCP, LangGraph or similar, Textual TUI	End-to-end diagnosis rate on injected faults; cost and latency per query
11	`nwdaf-lite` KPI anomaly detection	Prometheus metrics, Grafana	scikit-learn, MLflow, K8s Deployment, GitHub Actions	Detection rate and false alarms per fault window
12	Reference architecture: GenAI-assisted NOC	All of the above	Decision records, diagrams	A document a Tier-1 operator architect could review

Technical references

- GSMA Open-Telco LLM Benchmarks
- AI/ML in 3GPP 5G Advanced, arXiv 2512.03728
- AgentxGCore, arXiv 2606.00417
- Agentic-NWDAF, EURECOM
- Model Context Protocol docs
- MLflow docs
- OWASP Top 10 for LLM Applications